

Ejercicio 1

Como ejecutarlo?

El ejercicio está hecho en C++ y contiene un archivo make para poder compilar fácilmente. La salida del make crea el archivo “ejercicio_1” en la ruta “/bin/ejercicio_1”.

Para poder generar input el proceso padre funciona como menú, se crean los “combos” usando las opciones del menú y al ir creando los pedidos se puede ver en los logs (/logs/) como se van pasando entre cada proceso. Para salir se usa la opción “3” del menú.

En el header “**cocina.hpp**” se encuentran los *define* para parametrizar cantidad de procesos hijos y tiempo de preparación de cada combo (Con este tiempo simulamos trabajo y probamos concurrencia).

En el header “**colaMemCompartida.hpp**” se encuentra el *define* que parametriza la cantidad de pedidos máxima.

Manejo de concurrencia

La clase **ColaMemCompartida** contiene punteros a los semáforos y a una cola. Se pueden implementar N instancias de la clase según la cantidad de colas compartidas se quieran crear, funciona de forma “genérica” con implementaciones de Push, Pop y Logueo, utilizando cada método los semáforos correspondientes para asegurar la sincronización, de esta forma se abstrae la sincronización para que los hijos solamente usen los métodos.

En el método Init se crea el área de memoria compartida para los semáforos y el área de memoria compartida para la cola.

Recurso	Semáforo
Acceso a cola	mutex
Cantidad de pedidos (consumidor)	items
Espacio en cola (productor)	espacio
Acceso archivo log	logMutex

Monitoreo

Para monitorear la concurrencia el sistema lleva la auditoría de los pedidos mediante

archivos de log. Cada proceso hijo accede a la instancia global de la cola que consume la cuál implementa el método `Log()` para ir guardando el estado del pedido.

Cada archivo de log contiene los siguientes datos:

- Timestamp
- PID
- ID Pedido
- Tipo combo
- Estado

Limpieza del sistema

El proceso padre recibe la señal de terminar, una vez recibida deja de recibir pedidos y ejecuta el destructor de la instancia de **Cocina** (clase que realizó el `fork` de todos los procesos hijos). El destructor de **Cocina** se encarga de ir llevando la señal de finalización a todos los procesos hijos, tiene un mapa con tipo de hijo + pid por lo que envía la señal primero a los hijos que reciben el pedido, espera a que finalicen y sigue iterando hasta que todos finalicen en orden, de esta forma nos aseguramos que todos los hijos terminen su trabajo antes de salir. Al hacer el `kill` del hijo se ejecuta el destructor de **ColaMemCompartida** el cuál contiene:

- `munmap()`: libera la memoria compartida.
- `sem_destroy()`: destruye los semáforos.

Finalmente el destructor de cocina espera a los hijos:

- `waitpid()`: asegura que no queden procesos zombies.

Herramientas utilizadas para validar el monitoreo de los procesos

Durante el desarrollo y prueba del sistema, se utilizaron distintas herramientas para observar y validar el comportamiento de los procesos utilizando herramientas de monitoreo:

`ps -ef`. Lista todos los procesos activos, incluyendo el proceso padre y los procesos hijos (cocineros) generados con `fork()`.

`ps -T -p <pid>`. Muestra todos los hilos dentro de un proceso, permitiendo verificar que cada cocinero es un proceso independiente.

Se usó para verificar que:

- Existen la cantidad de hijos definidos en el archivo de header: **cocina.hpp**
- Todos los procesos hijos se crean correctamente.
- No quedan procesos zombies tras finalizar la jornada.

htop - Al ser una herramienta para visual, se utilizó para monitorear:

- Uso de CPU y memoria de cada proceso.
- Visualización en árbol para observar la relación padre-hijo entre el proceso principal y los cocineros.

Isof - Nos ha permitido ver los archivos y recursos que el proceso tiene abiertos. Fue muy útil para detectar si quedaron archivos sin cerrar.

Salidas en consola para seguimiento funcional. Ha permitido validar que los cocineros procesan los pedidos correctamente y ver si hay pedidos que quedaron sin atender, para no ser molesto muchas salidas por consola se retiraron ya que en el log se encuentra todo lo que se necesita.

Ejercicio 2

Interacción Cliente-Servidor (flujo de mensajes)

El ejercicio está hecho en C++ y contiene un archivo make en la carpeta “servidor”, y otro en la carpeta “cliente” para poder compilarlos fácilmente.

- La salida del make del “servidor” crea el archivo “ejercicio_2_servidor”, en la ruta “/bin/ejercicio_2_servidor”
- La salida del make de “cliente” crea el archivo “ejercicio_2_cliente” en la ruta “/bin/ejercicio_2_cliente”.

Para poder simular la llegada de pedidos se debe inicializar el servidor. Una vez en funcionamiento, los clientes que se conecten a ese servidor van a poder seleccionar mediante un menú los distintos tipos de combos (simple, doble o completo). Luego, cuando el cliente seleccione un combo, le va a mandar un mensaje al servidor con su pedido. Este lo va a recibir, y va a procesarlo para después entregarle el pedido “completo” al cliente. El cliente va a recibir la respuesta del servidor y va a dejar de “escuchar” al servidor.

- Cliente:
 - Se conecta al servidor.
 - Envía su pedido ({"tipo": "pedido", "combo": "S"}).
 - Escucha mensajes del servidor mostrando el avance del combo.
 - Al recibir {"estado": "entregado"}, puede cerrar la conexión enviando {"tipo": "salir"}.
- Servidor:
 - Al aceptar una conexión, lanza un hilo (cocinero).
 - Lee el pedido y simula el flujo de etapas.
 - Envía mensajes JSON para cada estado.
 - Si hay más de 5 clientes activos, puede enviar: {"estado": "ocupado", "mensaje": "Cocina llena, espere un momento"}
 - Si el cliente se desconecta, el hilo finaliza y libera el recurso.

Manejo de concurrencia

- Control de cantidad de conexiones con un contador `clientesActivos` y `mutex`.
- Los cocineros consumen pedidos de la cola con `mutex` + `condition_variable`.
- Hilos de cliente separados solo reciben y encolan pedidos.

Manejo de errores

- Clientes que se desconectan: detectados por `recv()`; el socket se cierra y el recurso se libera.
- Evita terminación por `SIGPIPE`: `signal(SIGPIPE, SIG_IGN)`.
- Logs: consola coloreada y archivo diario (`logs_YYYY-MM-DD.txt`).
- Reinicio limpio: uso de `SO_REUSEADDR` + handler de `SIGINT`.

Apagado automático

Cuando:

- No hay clientes activos (`clientesActivos == 0`)
- La cola de pedidos está vacía

Entonces:

- Se cierra el socket servidor.
- Se notifica a todos los hilos para finalizar.
- Se imprime mensaje de apagado y se liberan los recursos.

Bibliotecas utilizadas

- `nlohmann/json`: parseo y construcción de mensajes JSON.
- Librerías estándar de C++ (`thread`, `mutex`, `chrono`, `condition_variable`, `queue`, `fstream`, `signal`, `sys/socket.h`).

Herramientas utilizadas para validar el monitoreo de los procesos

Durante el desarrollo y prueba del sistema, se utilizaron distintas herramientas para observar y validar el comportamiento de los procesos de servidor y cliente, y los hilos involucrados en la ejecución.

`htop`. Aunque su visualización es un poco más dificultosa, se utilizó `htop` para:

1. Monitorear en tiempo real los procesos creados por el servidor y los clientes.
2. Confirmar que cada cliente lanza un proceso nuevo o que el servidor mantiene sus 5 hilos cocineros activos.
3. Ver el uso de CPU y memoria durante la simulación de múltiples pedidos.

`ps + grep`. Para listar procesos activos. Nos ha permitido validar cuántos hilos estaban corriendo y si se cerraban correctamente al finalizar la cocina.

Ejemplo de uso:

`ps aux | grep servidor`

`ps -T -p <pid>`

```
malaz@LAPTOP-644TVNPS:~$ ps aux | grep servidor
malaz  14626  0.0  0.0 16208 1288 pts/0    S   23:56   0:00 grep --color=auto servidor
```

`lsof -i :8080`. Para ver las conexiones activas en el puerto. Nos ha ayudado a verificar si los sockets TCP se abrían correctamente y si se cerraban luego de completar los pedidos.

Ejemplo de uso:

`lsof -i :8080`

`ss` o `netstat`. Para inspeccionar puertos en escucha. Así es como confirmamos que el servidor estaba escuchando correctamente en el puerto 8080.

`ss -ltnp | grep 8080`

Archivos de log (`logs_YYYY-MM-DD.txt`)

El sistema también implementa su propio registro de actividad por cocinero, lo cual fue un excelente complemento a las herramientas / comandos del sistema operativo que hemos utilizado previamente.